



Universidade de Brasília – UnB
Faculdade UnB Gama – FGA
Engenharia de Software

**Aplicação de Engenharia de Software em
Sistemas Robóticos de Reabilitação: estudos de
caso em realidade virtual e ciclismo com
eletroestimulação funcional no Projeto EMA**

Autores: Genilson Silva de Araújo Júnior
Helder Lourenço de Abreu Marques

Orientador: Prof. Dr. Roberto de Souza Baptista

Brasília, DF
2026



Genilson Silva de Araújo Júnior
Helder Lourenço de Abreu Marques

**Aplicação de Engenharia de Software em Sistemas
Robóticos de Reabilitação: estudos de caso em realidade
virtual e ciclismo com eletroestimulação funcional no
Projeto EMA**

Monografia submetida ao curso de graduação
em Engenharia de Software da Universidade
de Brasília, como requisito parcial para ob-
tenção do Título de Bacharel em Engenharia
de Software.

Universidade de Brasília – UnB
Faculdade UnB Gama – FGA

Orientador: Prof. Dr. Roberto de Souza Baptista

Brasília, DF

2026

Araújo Júnior, Genilson Silva de

Aplicação de Engenharia de Software em Sistemas Robóticos de Reabilitação: estudos de caso em realidade virtual e ciclismo com eletroestimulação funcional no Projeto EMA / Genilson Silva de Araújo Júnior, Helder Lourenço de Abreu Marques. – Brasília, DF, 2026-

53 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. Roberto de Souza Baptista

Trabalho de Conclusão de Curso – Universidade de Brasília – UnB
Faculdade UnB Gama – FGA , 2026.

1. Engenharia de Software. 2. Robótica de Reabilitação. I. Prof. Dr. Roberto de Souza Baptista. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. Aplicação de Engenharia de Software em Sistemas Robóticos de Reabilitação: estudos de caso em realidade virtual e ciclismo com eletroestimulação funcional no Projeto EMA

CDU 004:005.6:615.8

Genilson Silva de Araújo Júnior
Helder Lourenço de Abreu Marques

Aplicação de Engenharia de Software em Sistemas Robóticos de Reabilitação: estudos de caso em realidade virtual e ciclismo com eletroestimulação funcional no Projeto EMA

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Trabalho aprovado. Brasília, DF, 15 de julho de 2026:

Prof. Dr. Roberto de Souza Baptista
Orientador

Prof. Dr. Renato Coral Sampaio
Convidado 1

Prof. Dr. Geovany Araújo Borges
Convidado 2

Brasília, DF
2026

Agradecimentos

Eu, Genilson Silva de Araújo Júnior, agradeço a Deus e à minha família — meus pais, meu irmão e minha irmã —, base sobre a qual me firmei para chegar até aqui, permanecer e nunca desistir. Cada prova superada e cada trabalho concluído foram por vocês.

À minha mãe, Irenildes Lacerda de Santana, que tudo fez pelos filhos e nunca deixou de ajudar, mesmo quando não podia; ao meu pai, Genilson Silva de Araújo, sempre com uma palavra de incentivo e de fé; e ao meu irmão, Anderson Santana de Araújo, que, mesmo à distância, foi apoio, conselho e espelho de admiração. À minha irmã, Nataly Santana de Araújo, minha parceira, que me deu forças, conselhos e a direção certa nos momentos decisivos: foi ela quem abriu a trilha que hoje percorro — se cheguei a Brasília para estudar, foi porque ela ousou primeiro. É por eles, e para eles, cada passo que já dei e cada um que ainda darei.

Eu, Helder Lourenço de Abreu Marques, agradeço aos meus pais, Francisco Marques Batista e Mirian Lourenço de Abreu, por me incentivarem e me darem todo o apoio para ser quem eu sou, por todo o sacrifício e esforço que fizeram para me ver, um dia, concluindo uma graduação, torcendo e motivando diariamente. Aos meus familiares e amigos, por estarem ao meu lado e por eu poder contar com eles em todos os momentos. Aos colegas de classe e aos professores, agradeço por toda a experiência e todo o conhecimento adquiridos. E agradeço até mesmo os momentos ruins, pois é sob pressão que os diamantes são forjados.

*“Aquele que pergunta pode ser um tolo por cinco minutos.
Aquele que deixa de perguntar será um tolo para o resto da vida.”
(Provérbio chinês)*

Resumo

O Projeto EMA, conduzido no Laboratório de Automação e Robótica (LARA) da Universidade de Brasília, desenvolve sistemas de reabilitação motora que integram eletroestimulação funcional, robótica e realidade virtual. Ao longo de anos, sucessivas gerações de estudantes produziram diversos subsistemas que, embora funcionais do ponto de vista da pesquisa, apresentam fragilidades recorrentes de Engenharia de Software: arquiteturas acopladas, ausência de documentação, versionamento desorganizado e uma lacuna sistemática entre o que está registrado e o que está de fato montado. Este trabalho, de natureza aplicada e exploratória, aplica o instrumental da Engenharia de Software ao diagnóstico, à documentação e à proposta de reestruturação desses sistemas, adotando uma postura de consultoria: assume-se a perspectiva de um novo integrante que tenta reproduzir os sistemas tal como se encontram, registrando cada obstáculo como dado de diagnóstico. A investigação organiza-se em dois estudos de caso complementares e em estágios opostos de maturidade: o sistema de realidade virtual imersivo, inativo há anos, cujo desafio é a reativação; e o sistema de ciclismo assistido por eletroestimulação funcional, ativo, porém de difícil reprodução. Para cada caso, documenta-se a arquitetura atual, relata-se a tentativa de reprodução e mapeiam-se as limitações específicas; em seguida, consolidam-se os problemas comuns, com destaque para a organização do versionamento e a documentação. Como resultado, o trabalho entrega o diagnóstico estruturado, a diagramação padronizada das arquiteturas e um conjunto de recomendações de reestruturação, preparando o terreno para a etapa de integração a ser conduzida em trabalho posterior.

Palavras-chave: engenharia de software. robótica de reabilitação. documentação. refatoração. projeto EMA.

Abstract

Project EMA, developed at the Automation and Robotics Laboratory (LARA) of the University of Brasília, builds motor rehabilitation systems that integrate functional electrical stimulation, robotics and virtual reality. Over the years, successive generations of students have produced several subsystems that, although functional from a research standpoint, exhibit recurring software engineering weaknesses: coupled architectures, lack of documentation, disorganized version control and a systematic gap between what is recorded and what is actually assembled. This applied, exploratory work applies software engineering methods to the diagnosis, documentation and restructuring proposal of these systems, adopting a consulting stance: it takes the perspective of a new member who tries to reproduce the systems as they are, recording every obstacle as diagnostic data. The investigation is organized around two complementary case studies at opposite maturity stages: the immersive virtual reality system, inactive for years, whose challenge is reactivation; and the functional electrical stimulation cycling system, active yet hard to reproduce. For each case, the current architecture is documented, the reproduction attempt is reported and the specific limitations are mapped; the common problems are then consolidated, with emphasis on version control organization and documentation. As a result, the work delivers a structured diagnosis, standardized architecture diagrams and a set of restructuring recommendations, preparing the ground for the integration stage to be carried out in future work.

Key-words: software engineering. rehabilitation robotics. documentation. refactoring. Project EMA.

Lista de ilustrações

Figura 1 – Visão geral do ecossistema do Projeto EMA e os dois estudos de caso deste trabalho.	15
Figura 2 – Continuum realidade–virtualidade de Milgram e Kishino.	26
Figura 3 – Formas de visualização do movimento em realidade virtual.	27
Figura 4 – Voluntário utilizando o sistema de realidade virtual do Projeto EMA. .	33
Figura 5 – Arquitetura do sistema de realidade virtual, conforme documentada na fonte.	34
Figura 6 – Sistema de ciclismo com FES: (a) montagem física; (b) arquitetura de controle em nós.	36
Figura 7 – Fluxo de versionamento proposto para a ramificação principal.	45

Lista de tabelas

Tabela 1 – Síntese das limitações identificadas por estudo de caso.	43
Tabela 2 – Repositórios do ecossistema EMA e o seu estado de documentação e versionamento.	53

Lista de abreviaturas e siglas

ADR	<i>Architecture Decision Record</i> (Registro de Decisão de Arquitetura)
AVC	Acidente Vascular Cerebral
CBSE	<i>Component-Based Software Engineering</i> (Engenharia de Software Baseada em Componentes)
DDS	<i>Data Distribution Service</i>
EMA	Estimulação Muscular Assistida (Projeto EMA)
FES	<i>Functional Electrical Stimulation</i> (Eletroestimulação Funcional)
HMD	<i>Head-Mounted Display</i> (Capacete de Realidade Virtual)
HMI	<i>Human-Machine Interface</i> (Interface Humano-Máquina)
IMU	<i>Inertial Measurement Unit</i> (Unidade de Medição Inercial)
LARA	Laboratório de Automação e Robótica
QoS	<i>Quality of Service</i> (Qualidade de Serviço)
ROS	<i>Robot Operating System</i>
RV	Realidade Virtual
TDD	<i>Test-Driven Development</i> (Desenvolvimento Orientado a Testes)
UnB	Universidade de Brasília

Sumário

1	INTRODUÇÃO	14
1.1	Contextualização	14
1.2	Os Dois Estudos de Caso	16
1.3	Problema de Pesquisa	16
1.4	Justificativa	17
1.5	Objetivos	18
1.5.1	Objetivo Geral	18
1.5.2	Objetivos Específicos	18
1.6	Metodologia	19
1.7	Organização do Trabalho	19
2	FUNDAMENTAÇÃO TEÓRICA	20
2.1	Engenharia de Software para Sistemas Embarcados e Robóticos	20
2.2	Refatoração	21
2.2.1	Composição de Métodos	21
2.2.2	Movimentação de Recursos entre Objetos	21
2.2.3	Simplificação de Expressões Condicionais	21
2.2.4	Organização de Dados	22
2.2.5	Quando (e Quando Não) Refatorar	22
2.3	Documentação Técnica	22
2.3.1	Arquitetura de Conteúdo: o Arcabouço Diátaxis	23
2.3.2	Documentação de Arquitetura: Modelo C4 e ADR	23
2.3.3	Documentação como Código e Documentação Viva	23
2.4	Versionamento e Organização de Repositórios	24
2.5	Arquitetura Baseada em Componentes e o ROS	24
2.6	Fundamentos de Domínio	25
2.6.1	Reabilitação de Marcha e Neuroplasticidade	25
2.6.2	Realidade Virtual, Aumentada e Mista	25
2.6.3	Realidade Virtual Aplicada à Reabilitação	26
2.6.4	Eletroestimulação Funcional e Ciclismo Assistido	27
2.6.5	Captura de Movimento e Sensoriamento Inercial	27
3	METODOLOGIA	29
3.1	Natureza e Abordagem da Pesquisa	29
3.2	A Postura de Consultoria	29
3.3	Fases da Investigação	29

3.4	Instrumentos e Fontes de Dados	30
3.5	Delimitação do Escopo	31
4	ESTUDO DE CASO A — REALIDADE VIRTUAL	32
4.1	Visão Geral do Sistema	32
4.2	Arquitetura Atual (As-Is)	33
4.3	Tentativa de Reprodução	35
4.4	Limitações Específicas Identificadas	35
5	ESTUDO DE CASO B — CICLISMO COM ELETROESTIMULA- ÇÃO FUNCIONAL	36
5.1	Visão Geral do Sistema	36
5.1.1	Dois Repositórios, Duas Gerações	37
5.2	Arquitetura Atual (As-Is)	37
5.2.1	Lacuna entre o Registrado e o Montado	38
5.3	Tentativa de Reprodução	38
5.4	Limitações Específicas Identificadas	39
6	DIAGNÓSTICO CONSOLIDADO	41
6.1	Organização do Versionamento e dos Repositórios	41
6.2	Documentação e Reprodutibilidade	42
6.3	Arquitetura e Acoplamento	42
6.4	Síntese: a Lacuna entre o Registrado e o Montado	43
7	RECOMENDAÇÕES E CAMINHO DE REESTRUTURAÇÃO	44
7.1	Padronização do Versionamento e dos Repositórios	44
7.2	Documentação Mínima e Onboarding	45
7.3	Diagramação Padronizada da Arquitetura	46
7.4	Modularização e Desacoplamento	46
7.5	Caminho para a Integração (Trabalho Futuro)	46
8	CONSIDERAÇÕES FINAIS	48
8.1	Contribuições	48
8.2	Limitações do Trabalho	48
8.3	Trabalhos Futuros	49
	REFERÊNCIAS	50

APÊNDICES	52
APÊNDICE A – INVENTÁRIO DOS REPOSITÓRIOS DO LABORATÓRIO NO GITHUB	53

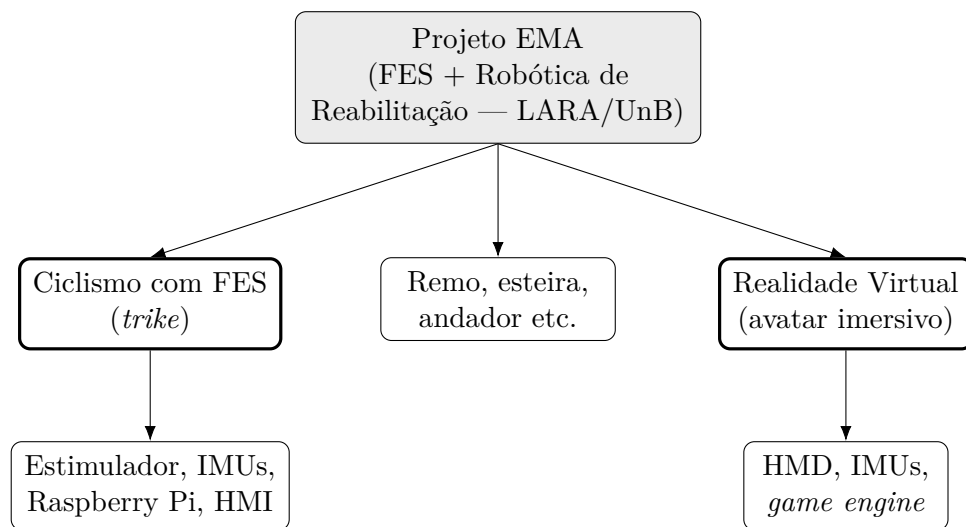
1 Introdução

1.1 Contextualização

A reabilitação motora de pessoas que perderam, parcial ou totalmente, a capacidade de locomoção é um dos grandes desafios contemporâneos da área da saúde. Condições neurológicas como o acidente vascular cerebral (AVC) e a lesão medular figuram entre as principais causas de incapacidade física de longo prazo, comprometendo a marcha, a independência e a qualidade de vida dos indivíduos acometidos ([RAJASHEKAR et al., 2024](#)). Há um consenso crescente de que a recuperação funcional depende de treinamento motor *intensivo, repetitivo e engajador*, capaz de estimular os mecanismos de reorganização do sistema nervoso ([LOUREIRO et al., 2024](#)). É nesse contexto que tecnologias assistivas como a robótica de reabilitação, a eletroestimulação funcional (FES) e a realidade virtual (RV) vêm sendo progressivamente incorporadas à prática clínica e à pesquisa acadêmica.

No Brasil, uma das iniciativas de referência nessa frente é o **Projeto EMA**, conduzido no Laboratório de Automação e Robótica (LARA) da Universidade de Brasília (UnB). O projeto investiga o uso da eletroestimulação funcional integrada a dispositivos robóticos — como triciclos, esteiras, equipamentos de remo e andadores — para a reabilitação motora e cardiovascular de pessoas com lesão nos membros inferiores. Ao longo de anos, sucessivas gerações de estudantes de graduação e pós-graduação desenvolveram, nesse ecossistema, uma coleção de subsistemas de hardware e software: nós de controle de estimuladores, drivers de sensores inerciais, interfaces de operação, ambientes virtuais imersivos e ferramentas de aquisição e análise de dados. A Figura 1 apresenta uma visão geral desse ecossistema e situa os dois subsistemas examinados neste trabalho.

Figura 1 – Visão geral do ecossistema do Projeto EMA e os dois estudos de caso deste trabalho.



Fonte: elaborado pelos autores.

Do ponto de vista da funcionalidade, muitos desses subsistemas cumpriram — e alguns ainda cumprem — o seu propósito de pesquisa: chegaram a funcionar, geraram resultados e sustentaram publicações. Entretanto, quando observados sob a ótica da *Engenharia de Software*, revelam um conjunto recorrente de fragilidades que compromete a sua continuidade: arquiteturas monolíticas e fortemente acopladas, ausência de documentação técnica e de guias de instalação, código sem padronização, repositórios de versionamento desorganizados e uma lacuna sistemática entre aquilo que está *registrado* (em artigos, diagramas e trabalhos anteriores) e aquilo que está de fato *montado e em operação* no laboratório. O resultado é que cada novo integrante que chega ao grupo enfrenta uma curva de reaproveitamento custosa, muitas vezes tendo de reconstruir, por tentativa e erro, conhecimento que deveria estar documentado.

É precisamente esse o objeto deste Trabalho de Conclusão de Curso. Diferentemente de um trabalho voltado a criar uma nova funcionalidade, a contribuição aqui proposta é de natureza metodológica e diagnóstica: aplicar o instrumental da Engenharia de Software para *compreender, documentar, diagramar e diagnosticar* o estado atual dos sistemas do Projeto EMA, apontando as suas limitações e propondo o caminho para que evoluam de protótipos de pesquisa frágeis para uma base de software sustentável. Para dar concretude a essa investigação, o trabalho organiza-se em torno de **dois estudos de caso** complementares, que representam dois estágios distintos de maturidade dentro do mesmo ecossistema.

1.2 Os Dois Estudos de Caso

O trabalho é desenvolvido em dupla e articula duas frentes que compartilham o mesmo laboratório, o mesmo orientador e os mesmos problemas estruturais de Engenharia de Software, mas que se encontram em estágios opostos do ciclo de vida:

- **Estudo de caso A — Realidade Virtual.** O sistema de RV imersivo desenvolvido para capturar o movimento do paciente por sensores inerciais e reproduzi-lo, em tempo real, em um avatar virtual (BALBINO, 2020). Trata-se de um sistema cujo último desenvolvimento registrado no repositório data de 2019 e que se encontra inativo desde então; o seu desafio central é a *reativação*: recuperar, a partir do que está registrado, um sistema que já funcionou, identificando o que falta para colocá-lo novamente em operação. Por envolver menos componentes de hardware, é o caso de mais rápida retomada.
- **Estudo de caso B — Ciclismo com Eletroestimulação Funcional (FES *Cycling*).** O sistema de controle da *trike* assistida por eletroestimulação, no qual sensores inerciais capturam a posição dos pedais e comandam os estimuladores que produzem o movimento de pedalada (BRINDEIRO, 2017; BAPTISTA et al., 2022). Diferentemente do caso anterior, é um sistema *ativo*, com equipe trabalhando sobre ele, porém marcado por documentação ausente e por dificuldades de reprodução. Por envolver mais componentes de hardware, é o caso de diagnóstico mais complexo.

A escolha por dois casos em estágios distintos é deliberada: permite contrastar os problemas de Engenharia de Software que surgem quando um sistema é *abandonado* com aqueles que persistem mesmo quando ele está *ativo*, revelando que a fragilidade estrutural independe do estado de atividade do projeto. Ao longo do texto, os problemas específicos de cada caso são tratados em capítulos próprios, enquanto os problemas comuns — notadamente os relativos ao versionamento e à documentação — são consolidados em um diagnóstico conjunto.

1.3 Problema de Pesquisa

Os sistemas do Projeto EMA foram concebidos, em sua maioria, como provas de conceito orientadas a demonstrar viabilidade técnica e a sustentar investigações científicas pontuais. Cumprido esse objetivo imediato, tais sistemas raramente receberam o tratamento de engenharia necessário para que se tornassem artefatos reutilizáveis: faltam-lhes documentação, padronização, modularidade e uma organização de versionamento que preserve o conhecimento produzido. O maior gargalo do laboratório, portanto, não está na

ausência de funcionalidade, mas na *forma como o software está estruturado, documentado e versionado*.

Esse cenário produz uma consequência prática mensurável: a dificuldade de *reproduzibilidade*. Confrontado com um repositório do laboratório, um novo integrante — ainda que tecnicamente competente — não encontra os elementos mínimos que a boa prática de Engenharia de Software prescreve para que um sistema possa ser instalado, executado e compreendido: um *README* com passo a passo, a especificação do ambiente, os diagramas de arquitetura coerentes com o que está de fato implementado e um histórico de versões inteligível. Como se demonstra ao longo deste trabalho, a tentativa de *clonar e executar* os sistemas tal como se encontram esbarra em obstáculos que vão de detalhes de configuração a lacunas estruturais, como código-fonte ausente e divergências entre o hardware montado e o hardware documentado.

Diante desse quadro, o presente trabalho organiza-se em torno da seguinte questão de pesquisa: *de que forma o instrumental da Engenharia de Software pode ser aplicado ao diagnóstico, à documentação e à reestruturação dos sistemas robóticos de reabilitação do Projeto EMA, de modo a reduzir a lacuna entre o que está registrado e o que está montado e a viabilizar que um novo integrante seja capaz de reproduzir, compreender e evoluir esses sistemas?*

1.4 Justificativa

A relevância deste trabalho manifesta-se em três dimensões complementares.

Do ponto de vista da **Engenharia de Software** — área de formação dos autores —, o laboratório oferece um cenário real, com legado e restrições concretos, no qual princípios consagrados da disciplina podem ser aplicados e avaliados. A separação de interesses, o baixo acoplamento, a documentação arquitetural e o versionamento disciplinado são reconhecidos como determinantes da manutenibilidade e da longevidade de sistemas de software (SOMMERVILLE, 2011). Atuar sobre esses aspectos, com postura de *arquiteto de software* — e não apenas de programador —, é precisamente onde um trabalho da área pode oferecer a maior contribuição a um laboratório historicamente centrado no hardware e na pesquisa aplicada.

Do ponto de vista **científico**, o Projeto EMA é uma iniciativa de longa duração, conduzida por sucessivas gerações de estudantes. A continuidade desse esforço depende criticamente da capacidade de cada nova geração compreender e construir sobre o trabalho das anteriores. Sem documentação e sem uma arquitetura inteligível, cada transição de equipe implica retrabalho e perda de conhecimento — um custo que a alta rotatividade característica de um laboratório acadêmico apenas amplifica. Estruturar, documentar e

diagnosticar os sistemas é, assim, condição para a sustentabilidade das próprias linhas de pesquisa.

Do ponto de vista **clínico**, os sistemas em questão são ferramentas com potencial terapêutico documentado. A literatura demonstra que a realidade virtual e a eletroestimulação funcional, sobretudo quando combinadas à robótica de reabilitação, podem aumentar a motivação e a adesão ao tratamento e contribuir para melhores desfechos motores (LOUREIRO et al., 2024; MACCHITELLA et al., 2023). Recuperar e consolidar essas ferramentas em bases de software confiáveis aproxima o laboratório de protocolos de reabilitação mais ricos, em benefício direto dos pacientes.

1.5 Objetivos

1.5.1 Objetivo Geral

Aplicar o instrumental da Engenharia de Software ao diagnóstico, à documentação e à proposta de reestruturação dos sistemas robóticos de reabilitação do Projeto EMA, tomando como estudos de caso o sistema de realidade virtual e o sistema de ciclismo com eletroestimulação funcional, de modo a reduzir a lacuna entre o que está registrado e o que está montado e a viabilizar a sua reprodução, compreensão e evolução.

1.5.2 Objetivos Específicos

Para o alcance do objetivo geral, definem-se os seguintes objetivos específicos:

- revisar a literatura sobre boas práticas de Engenharia de Software aplicáveis a sistemas embarcados e robóticos, com ênfase em refatoração, documentação técnica, versionamento e arquitetura baseada em componentes;
- levantar e documentar a arquitetura atual (*as-is*) de cada um dos dois sistemas, reconstruindo, por análise documental e de código, os seus componentes e fluxos de dados;
- confrontar o que está registrado (artigos, diagramas e trabalhos anteriores) com o que está de fato montado e em operação, mapeando as lacunas e as incoerências;
- diagnosticar as limitações de Engenharia de Software comuns aos dois sistemas, em especial as relativas à organização dos repositórios, à documentação e à reprodutibilidade;
- produzir os modelos e diagramas de arquitetura padronizados que representem os sistemas de forma coerente;

- propor recomendações e um caminho de reestruturação — de padronização de repositórios e documentação a modularização e desacoplamento — que preparem o terreno para a etapa de integração a ser conduzida em trabalho posterior.

1.6 Metodologia

Quanto à sua natureza, este trabalho caracteriza-se como uma *pesquisa aplicada*, de caráter *exploratório* e abordagem predominantemente *qualitativa*, voltada à compreensão e à reestruturação de sistemas de software existentes em um contexto real de pesquisa. Adota-se uma postura de *consultoria de Engenharia de Software*: os autores posicionam-se como engenheiros que, ao chegarem ao laboratório, recebem acesso aos repositórios e à infraestrutura e se propõem a reproduzir os sistemas tal como se encontram, registrando sistematicamente os obstáculos e as lacunas encontrados. O detalhamento dessa abordagem é apresentado no Capítulo 3.

1.7 Organização do Trabalho

Além desta introdução, o trabalho está organizado em mais sete capítulos. O Capítulo 2 apresenta a *fundamentação teórica*, começando pelas boas práticas de Engenharia de Software para sistemas embarcados — refatoração, documentação técnica, versionamento e arquitetura baseada em componentes (ROS) — e delimitando, em seguida, os fundamentos de domínio (reabilitação, realidade virtual, eletroestimulação e sensoramento inercial). O Capítulo 3 descreve a *metodologia da consultoria* adotada para o diagnóstico. Os Capítulos 4 e 5 apresentam, respectivamente, os *estudos de caso* da realidade virtual e do ciclismo com eletroestimulação, com a documentação da arquitetura atual, as tentativas de reprodução e as limitações específicas de cada um. O Capítulo 6 consolida o *diagnóstico dos problemas comuns*, com destaque para a organização do versionamento e a documentação. O Capítulo 7 reúne as *recomendações e o caminho de reestruturação* propostos. Por fim, o Capítulo 8 traz as *considerações finais*, retomando as contribuições, as limitações e os trabalhos futuros.

2 Fundamentação Teórica

Este capítulo reúne os fundamentos que sustentam o trabalho, organizados do geral para o específico, conforme a natureza da contribuição proposta. Parte-se das boas práticas de Engenharia de Software aplicáveis a sistemas embarcados e robóticos (Seção 2.1), passando pela refatoração (Seção 2.2), pela documentação técnica (Seção 2.3), pelo versionamento e organização de repositórios (Seção 2.4) e pela arquitetura baseada em componentes materializada no ROS (Seção 2.5). Por fim, delimitam-se os fundamentos de domínio (Seção 2.6) — reabilitação, realidade virtual, eletroestimulação e sensoriamento inercial — na medida necessária para compreender os dois estudos de caso.

2.1 Engenharia de Software para Sistemas Embarcados e Robóticos

A Engenharia de Software é a disciplina que se ocupa da produção de software de qualidade, dentro de prazos e custos previsíveis, ao longo de todo o seu ciclo de vida (SOMMERVILLE, 2011). Um princípio central da disciplina é o de que o custo dominante de um sistema não está na sua construção inicial, mas na sua *evolução* — as sucessivas alterações necessárias para corrigir defeitos, adaptar-se a novos ambientes e incorporar requisitos. Atributos como manutenibilidade, modularidade e baixo acoplamento, portanto, não são refinamentos estéticos, mas determinantes diretos do custo e da viabilidade de manter um sistema vivo ao longo do tempo.

Esses princípios assumem contornos particulares quando o software é *embarcado* e *robótico*, como é o caso dos sistemas do Projeto EMA. Nesse domínio, o software não existe isoladamente: ele controla e é controlado por hardware — sensores, atuadores, microcontroladores — sujeito a restrições de tempo real, de segurança e de recursos computacionais limitados. Um comando enviado a um estimulador elétrico ou a leitura de um sensor inercial têm consequências físicas imediatas, o que eleva a criticidade de falhas. Ao mesmo tempo, sistemas robóticos são intrinsecamente distribuídos e heterogêneos, o que torna a modularidade e a padronização das interfaces entre componentes uma necessidade, e não uma opção.

Um agravante recorrente em laboratórios de pesquisa é a tensão entre a cultura de prototipagem rápida — orientada a demonstrar viabilidade e produzir resultados publicáveis — e as boas práticas de engenharia que garantem a longevidade do software. O protótipo que “funciona uma vez” cumpre o seu papel científico imediato, mas, na ausência de documentação, testes e organização, converte-se em um passivo técnico (*technical debt*) que onera todas as gerações seguintes. As seções a seguir apresentam o instrumental

que este trabalho mobiliza para diagnosticar e reduzir esse passivo: refatoração, documentação, versionamento e arquitetura baseada em componentes.

2.2 Refatoração

Refatoração é o processo de alterar a estrutura interna de um software sem modificar o seu comportamento externo observável, com o objetivo de torná-lo mais simples, legível e manutenível (FOWLER, 2018). Trata-se de uma disciplina de melhoria contínua: em vez de uma reescrita monolítica e arriscada, aplicam-se transformações pequenas e verificáveis que preservam a funcionalidade enquanto aprimoram o desenho. Cada projeto possui características próprias, mas um repertório consagrado de técnicas orienta essas transformações; a seguir, destacam-se as mais pertinentes ao contexto deste trabalho.

2.2.1 Composição de Métodos

Um primeiro grupo de técnicas organiza o corpo das funções. *Extrair Método* (*Extract Method*) transforma um fragmento de código coeso em um método próprio, com nome descritivo, elevando a legibilidade e favorecendo o reúso; *Embutir Método* (*Inline Method*) faz o inverso, eliminando indireções desnecessárias quando o corpo de um método é tão evidente quanto o seu nome. De forma análoga, *Extrair Variável* (*Extract Variable*) atribui a uma variável bem nomeada o resultado de uma expressão complexa, tornando explícita a sua intenção.

2.2.2 Movimentação de Recursos entre Objetos

Quando as responsabilidades estão mal distribuídas, técnicas de movimentação reequilibram o desenho. *Mover Método/Campo* (*Move Method/Field*) desloca um membro para a classe que dele mais se beneficia; *Extrair Classe* (*Extract Class*) cria uma nova classe quando uma única passa a acumular responsabilidades que seriam de duas. Esse princípio de agrupar responsabilidades afins é diretamente análogo à forma como o ROS organiza a computação em *nós* especializados, cada qual responsável por uma parte bem delimitada do sistema.

2.2.3 Simplificação de Expressões Condicionais

A complexidade condicional é uma fonte frequente de defeitos. *Decompor Condicional* (*Decompose Conditional*) extrai a condição e as ações de ramificações complexas para métodos nomeados; *Consolidar Expressão Condicional* unifica verificações sucessivas que conduzem ao mesmo resultado; *Substituir Condicional por Polimorfismo* troca seleções por tipo (por exemplo, um `switch`) por herança e polimorfismo; e as *Cláusulas de*

Guarda (*Guard Clauses*) tratam casos excepcionais com retornos antecipados, mantendo limpo o “caminho feliz” da função. Em Python — linguagem predominante nos nós do EMA —, tais técnicas reduzem o aninhamento e o excesso de condicionais, alinhando-se à ênfase da linguagem em legibilidade.

2.2.4 Organização de Dados

Por fim, técnicas de organização de dados aumentam a clareza e a segurança do código. *Substituir Número Mágico por Constante* elimina valores literais soltos em favor de constantes descritivas, e *Encapsular Campo* restringe o acesso direto a atributos por meio de métodos, controlando as invariantes do objeto. Em sistemas de reabilitação, nos quais parâmetros como limiares de corrente e faixas angulares têm significado clínico preciso, nomear e encapsular esses valores é também uma medida de segurança.

2.2.5 Quando (e Quando Não) Refatorar

A refatoração é mais eficaz quando integrada à rotina de desenvolvimento, e não tratada como um evento isolado. A *Regra do Escoteiro* sintetiza essa postura: deixar o código mais limpo do que se o encontrou. Oportunidades naturais surgem ao adicionar funcionalidades, ao corrigir defeitos e durante revisões de código. Um pressuposto importante, contudo, é a existência de uma rede de segurança que garanta a preservação do comportamento: sem testes automatizados, refatorar aproxima-se de “mudar o código e torcer”. O ciclo de Desenvolvimento Orientado a Testes (*Test-Driven Development*, TDD) — escrever um teste que falha, fazê-lo passar com o código mais simples possível e, então, refatorar (BECK, 2003) — oferece essa rede. Há, ainda, situações em que refatorar é desaconselhável: nas proximidades de um prazo, pelo risco de introduzir defeitos; e quando o código está tão comprometido, ou desprovido de testes, que a reescrita se torna preferível — afinal, a refatoração pressupõe que o sistema atual funcione.

2.3 Documentação Técnica

Para os sistemas em estudo, a documentação é a maior carência isolada. Historicamente, adotou-se no laboratório a premissa implícita de que “o código é a documentação” — premissa que se revela insuficiente quando um novo integrante precisa instalar, executar e compreender o sistema a partir do zero. Esta seção apresenta os arcabouços que orientam uma documentação intencional e sustentável.

2.3.1 Arquitetura de Conteúdo: o Arcabouço Diátaxis

O arcabouço *Diátaxis* organiza a documentação técnica em quatro categorias definidas pela necessidade do usuário (PROCIDA, 2023): *tutoriais*, orientados ao aprendizado, que conduzem o iniciante por um percurso passo a passo; *guias de como fazer* (*how-to guides*), orientados a resolver tarefas específicas; *referência*, orientada à informação precisa (parâmetros, APIs, formatos de dados); e *explicação*, orientada à compreensão dos conceitos e das decisões de projeto. A distinção é útil porque cada categoria falha quando mistura propósitos — um tutorial sobrecarregado de detalhes de referência, por exemplo, deixa de cumprir o seu papel didático. Para um laboratório com alta rotatividade, os tutoriais e os guias de instalação são precisamente os artefatos cuja ausência mais onera a integração de novos membros.

2.3.2 Documentação de Arquitetura: Modelo C4 e ADR

A comunicação da arquitetura beneficia-se de notações padronizadas. O *Modelo C4* propõe descrever a arquitetura em quatro níveis de abstração progressivos — Contexto, Contêineres, Componentes e Código —, permitindo “dar zoom” do panorama geral ao detalhe conforme a audiência (BROWN, 2024). Complementarmente, os *Registros de Decisão de Arquitetura* (*Architecture Decision Records*, ADR) documentam, de forma incremental e datada, as decisões arquiteturais relevantes, o contexto em que foram tomadas e as suas consequências (NYGARD, 2011). Os ADR são especialmente valiosos em projetos de longa duração, pois preservam o *porquê* das escolhas — conhecimento que, de outro modo, se perde a cada transição de equipe.

2.3.3 Documentação como Código e Documentação Viva

Duas abordagens operacionais aumentam a probabilidade de a documentação permanecer atualizada. A *documentação como código* (*docs-as-code*) trata os artefatos de documentação com o mesmo rigor do código-fonte: escritos em linguagens de marcação leve (como Markdown), versionados em Git, submetidos a revisão por pares e publicados por pipelines automatizados. A *documentação viva* (*living documentation*) gera parte da documentação diretamente a partir do código ou de testes executáveis, reduzindo o risco de defasagem. Ambas convergem para um mesmo objetivo: fazer da documentação um subproduto natural do fluxo de trabalho, e não uma tarefa avulsa e facilmente adiada. Incorporar a atualização da documentação à *Definição de Pronto* (*Definition of Done*) de cada tarefa é uma prática de governança que reforça essa cultura.

2.4 Versionamento e Organização de Repositórios

O sistema de controle de versão é, em muitos projetos acadêmicos, a principal — por vezes a única — memória institucional do software. Ferramentas como o Git, e plataformas como o GitHub, oferecem recursos que vão muito além do simples armazenamento de arquivos: organização por equipes, rastreamento de tarefas e defeitos (*issues*), fluxos de revisão por *pull request*, marcação de versões (*releases*) e integração contínua. O uso disciplinado desses recursos — convenções de nomenclatura de repositórios, presença de arquivos README e de licença, histórico de *commits* inteligível e separação clara entre código ativo e arquivado — é o que permite que o conhecimento produzido seja localizável e reaproveitável.

A ausência dessas convenções produz o quadro oposto: repositórios duplicados para o mesmo projeto, nomes que não comunicam propósito, misturas entre trabalhos originais e cópias, e código sem qualquer ponto de entrada documentado. Como se detalha no Capítulo 6, esse é exatamente o cenário observado na organização do laboratório, e constitui uma das lacunas de Engenharia de Software mais custosas — justamente por ser das mais simples de corrigir com boas práticas.

2.5 Arquitetura Baseada em Componentes e o ROS

A Engenharia de Software Baseada em Componentes (*Component-Based Software Engineering*, CBSE) propõe construir sistemas pela composição de partes independentes e substituíveis, que interagem por meio de interfaces bem definidas. O *Robot Operating System* (ROS) é a materialização desse paradigma no domínio da robótica: não um sistema operacional no sentido estrito, mas um *middleware* que organiza a computação em nós — processos especializados e independentes — que se comunicam por um modelo de publicação/assinatura (*publish/subscribe*) sobre tópicos nomeados (Open Robotics, 2024). Essa arquitetura favorece diretamente o baixo acoplamento: um nó pode ser desenvolvido, testado, substituído ou reescrito sem afetar os demais, desde que preserve o contrato das mensagens que troca.

O ROS 2, adotado como alvo pelo laboratório, assenta a sua comunicação sobre o padrão *Data Distribution Service* (DDS). Dessa escolha decorrem duas propriedades relevantes para sistemas de reabilitação. A primeira é a *descoberta automática* de nós: sem a necessidade de um nó mestre central, os participantes localizam-se na rede e passam a comunicar-se de forma descentralizada. A segunda são as políticas de *Qualidade de Serviço* (*Quality of Service*, QoS), que permitem ajustar a comunicação à criticidade de cada fluxo de dados. Assim, a telemetria de um sensor inercial, contínua e tolerante a perdas, pode ser configurada como *best effort* (privilegiando a velocidade), ao passo que um comando enviado ao estimulador elétrico, que não pode ser perdido, é configurado

como *reliable* (garantindo a entrega). Sobre esses mesmos mecanismos é possível construir salvaguardas de segurança — por exemplo, associar um tempo de validade (*lifespan*) e sinais de vivacidade (*liveliness*) aos comandos críticos, de modo que o driver do estimulador entre em modo seguro caso deixe de receber comandos válidos do nó de controle. Esses conceitos são retomados na análise do estudo de caso do ciclismo com eletroestimulação (Capítulo 5).

2.6 Fundamentos de Domínio

Delimitados os fundamentos de Engenharia de Software, esta seção apresenta o domínio de aplicação na extensão necessária para compreender os dois estudos de caso: a reabilitação de marcha e a neuroplasticidade, a realidade virtual e a sua aplicação terapêutica, a eletroestimulação funcional aplicada ao ciclismo e a captura de movimento por sensores inerciais.

2.6.1 Reabilitação de Marcha e Neuroplasticidade

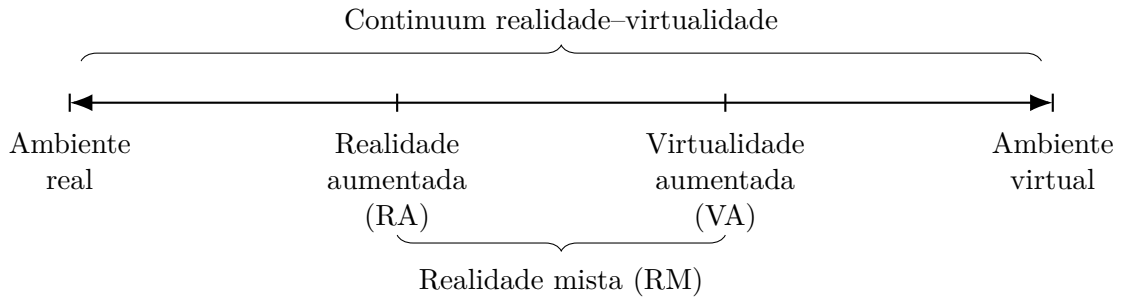
A marcha humana é uma tarefa motora complexa, resultante da coordenação fina entre o sistema nervoso central, o sistema musculoesquelético e a integração contínua de informações sensoriais. Lesões neurológicas como o AVC e a lesão medular interrompem, em graus variados, o controle dessa tarefa. A reabilitação de marcha busca restaurar essa capacidade explorando um princípio fundamental do sistema nervoso: a sua plasticidade. Entende-se por *neuroplasticidade* a capacidade do sistema nervoso de reorganizar a sua estrutura e a sua função em resposta à experiência. Kleim e Jones (2008) sistematizam esse conhecimento em princípios da plasticidade dependente de experiência, entre os quais a especificidade do treino, a importância da repetição e da intensidade e a relevância da saliência do estímulo. Esses princípios explicam *por que* um treino motor intensivo, repetitivo e engajador promove recuperação funcional (TAKEUCHI; IZUMI, 2013). É justamente sobre esses fatores — repetição, intensidade e saliência — que operam as tecnologias assistivas discutidas a seguir (RAJASHEKAR et al., 2024; MACCHITELLA et al., 2023).

2.6.2 Realidade Virtual, Aumentada e Mista

A realidade virtual (RV) pode ser definida como o uso de computação gráfica interativa para gerar ambientes tridimensionais nos quais o usuário percebe estar imerso e com os quais interage em tempo real. Dois conceitos caracterizam essa experiência: segundo Slater e Wilbur (1997), a *imersão* é uma propriedade objetiva do sistema — o grau em que os dispositivos substituem os estímulos reais por estímulos sintéticos —, ao passo que a *presença* é o estado subjetivo resultante, a sensação de *estar* no ambiente

virtual. Para situar a RV em relação a tecnologias correlatas, é clássica a taxonomia de [Milgram e Kishino \(1994\)](#), que organiza as combinações entre elementos reais e virtuais ao longo de um *continuum realidade–virtualidade*, ilustrado na Figura 2.

Figura 2 – Continuum realidade–virtualidade de Milgram e Kishino.



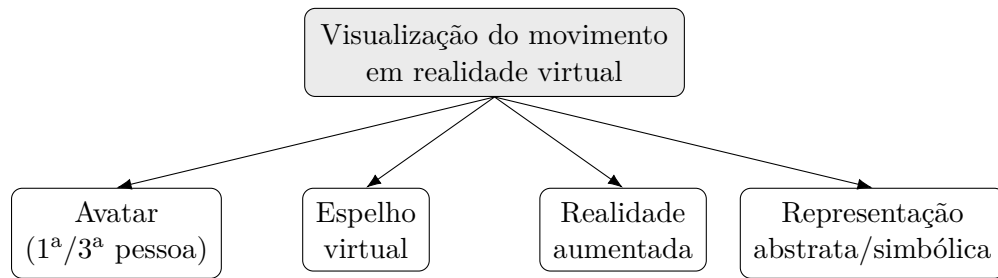
Fonte: elaborado pelos autores com base em [Milgram e Kishino \(1994\)](#).

Embora concebida antes da difusão dos dispositivos atuais, essa taxonomia permanece a referência conceitual dominante, ainda que trabalhos recentes proponham revisá-la à luz da evolução tecnológica ([SKARBEZ; SMITH; WHITTON, 2021](#)). Para os fins deste trabalho, o continuum de [Milgram e Kishino \(1994\)](#) é suficiente para posicionar o sistema em estudo: um ambiente virtual imersivo, no extremo virtual do continuum, no qual o movimento real do paciente é capturado e reproduzido.

2.6.3 Realidade Virtual Aplicada à Reabilitação

A aplicação da RV à reabilitação motora fundamenta-se na possibilidade de oferecer, em ambiente seguro e controlado, prática intensiva e repetitiva associada a realimentação sensorial enriquecida — precisamente os fatores associados à neuroplasticidade. A magnitude desse benefício deve ser avaliada com rigor: a revisão sistemática Cochrane de [Laver et al. \(2017\)](#) conclui que a RV não é superior à terapia convencional quando empregada isoladamente, mas mostra-se benéfica como *adjuvante*, com efeitos mais consistentes sob maior dose de treino. Um aspecto determinante é a forma como o sistema apresenta ao usuário o seu próprio movimento — a *visualização do movimento* —, cujas principais formas, identificadas por [Ferreira dos Santos et al. \(2016\)](#), a Figura 3 sintetiza.

Figura 3 – Formas de visualização do movimento em realidade virtual.



Fonte: elaborado pelos autores com base em [Ferreira dos Santos et al. \(2016\)](#).

O sistema de RV do Projeto EMA, analisado no Capítulo 4, adota a estratégia baseada em *avatar*, reproduzindo em tempo real o movimento dos membros do paciente capturado por sensores inerciais. A evidência sobre marcha e equilíbrio reforça o potencial da abordagem quando associada a dispositivos de treino locomotor ([MIRELMAN et al., 2016](#)), ainda que os efeitos sobre parâmetros de marcha permaneçam inconsistentes ([LEE et al., 2024](#)). Em cenários de maior comprometimento, protocolos que associaram RV imersiva, realimentação tátil e exoesqueletos controlados por interface cérebro-máquina chegaram a induzir recuperação neurológica parcial em pacientes com lesão medular crônica ([DONATI et al., 2016](#)).

2.6.4 Eletroestimulação Funcional e Ciclismo Assistido

A eletroestimulação funcional (*Functional Electrical Stimulation*, FES) consiste na aplicação de pulsos elétricos a músculos paralisados ou enfraquecidos, provocando contrações artificiais capazes de produzir movimento funcional. É a única técnica capaz de restaurar movimento em músculos paralisados, permitindo, por exemplo, que indivíduos com paraplegia executem o movimento de pedalada em uma bicicleta reclinada ([BAPTISTA et al., 2022](#)). No ciclismo assistido por FES, sensores medem a posição angular dos pedais e, a partir dela, determinam quais grupos musculares estimular e em que instante, de modo a produzir uma pedalada coordenada. O grupo do LARA acumula experiência consolidada nesse tema, inclusive com participação na competição internacional CYBATHLON, tendo desenvolvido desde o software embarcado de controle do triciclo ([BRINDEIRO, 2017](#)) até aprimoramentos de projeto centrados no usuário e técnicas de estimulação espacialmente distribuídas ([BAPTISTA et al., 2022](#)). Esse sistema constitui o segundo estudo de caso deste trabalho.

2.6.5 Captura de Movimento e Sensoriamento Inercial

Ambos os estudos de caso dependem da captura do movimento do paciente por *unidades de medição inercial* (*Inertial Measurement Units*, IMU) — sensores que combinam

acelerômetros, giroscópios e, frequentemente, magnetômetros para estimar a orientação de um segmento corporal ou de um componente mecânico no espaço. No sistema de RV, as IMUs fixadas aos membros do paciente — superiores e inferiores — fornecem as orientações que animam, em tempo real, os membros do avatar; no sistema de ciclismo, as IMUs estimam a posição angular dos pedais que orienta a máquina de estados da estimulação. Em ambos os casos, o sensoriamento inercial é a ponte entre o movimento físico real e a lógica de software — razão pela qual a integridade e a documentação dos seus drivers são críticas para a reprodução dos sistemas.

3 Metodologia

Este capítulo descreve a abordagem metodológica adotada para investigar os sistemas do Projeto EMA. A estratégia central é deliberadamente simples e reproduzível: assumir a perspectiva de um novo integrante — ou de um consultor de Engenharia de Software — que, ao chegar ao laboratório, recebe acesso aos repositórios e à infraestrutura e se propõe a *colocar os sistemas para funcionar tal como se encontram*, registrando sistematicamente cada obstáculo, lacuna e incoerência encontrados no caminho.

3.1 Natureza e Abordagem da Pesquisa

Quanto à sua natureza, este trabalho caracteriza-se como uma *pesquisa aplicada*, pois se volta à solução de um problema concreto de um contexto real. Quanto aos objetivos, é *exploratória*, uma vez que busca reconstruir e compreender sistemas pouco documentados. Quanto à abordagem, é predominantemente *qualitativa*, apoiada na análise documental, na leitura de código e na observação direta do hardware e das tentativas de execução. A unidade de análise são os dois estudos de caso descritos na Seção 1.2, tratados segundo a lógica de *estudo de caso múltiplo*: cada sistema é investigado em profundidade e, ao final, os achados são confrontados para distinguir o que é específico de cada um do que é comum a ambos.

3.2 A Postura de Consultoria

A escolha de uma postura de consultoria orienta todas as etapas seguintes e tem uma consequência metodológica importante: o valor da investigação não está apenas no que *funciona*, mas sobretudo no que *falha*. Cada dificuldade encontrada ao tentar reproduzir um sistema — um comando que não executa, um arquivo ausente, um diagrama que não corresponde ao hardware — é tratada como um *dado*, e não como um contratempo a ser silenciosamente contornado. Essa inversão de perspectiva é o que transforma a frustração de “o projeto não roda” em um diagnóstico estruturado e acionável. Pergunta-se, a cada passo: *o que um novo integrante encontraria aqui, e o que lhe faltaria?*

3.3 Fases da Investigação

O trabalho foi organizado nas seguintes fases metodológicas, aplicadas a cada estudo de caso:

1. **Revisão bibliográfica.** Levantamento e leitura crítica da literatura sobre boas práticas de Engenharia de Software (refatoração, documentação, versionamento, arquitetura baseada em componentes) e sobre o domínio de aplicação (reabilitação, RV, FES, sensoriamento inercial), consolidados no Capítulo 2.
2. **Levantamento documental.** Exame do que está *registrado* sobre cada sistema: trabalhos acadêmicos anteriores, diagramas, artigos e o conteúdo dos repositórios. Dessa etapa resulta o retrato de como o sistema *deveria* ser, segundo a documentação disponível.
3. **Tentativa de reprodução.** Clonagem dos repositórios e tentativa de instalar e executar os sistemas tal como se encontram, na infraestrutura do laboratório e em plataformas de teste. Cada obstáculo é registrado com precisão (comando, mensagem de erro, causa provável).
4. **Análise de código e engenharia reversa.** Leitura do código-fonte para reconstruir a arquitetura efetivamente implementada (*as-is*): componentes, nós, tópicos, fluxos de dados e pontos de acoplamento.
5. **Confronto entre o registrado e o montado.** Comparação sistemática entre a arquitetura documentada e a arquitetura implementada, e entre o hardware diagramado e o hardware fisicamente montado, com o objetivo de mapear lacunas e incoerências.
6. **Modelagem e diagramação.** Representação padronizada das arquiteturas por meio de diagramas coerentes com o que foi efetivamente observado, corrigindo ou completando os diagramas preexistentes quando necessário.
7. **Diagnóstico e proposta.** Síntese das limitações identificadas — específicas e comuns — e elaboração de recomendações e de um caminho de reestruturação, apresentados nos Capítulos 6 e 7.

3.4 Instrumentos e Fontes de Dados

As fontes de dados incluem: os repositórios da organização **lara-unb** no GitHub; os trabalhos acadêmicos de referência que documentam cada sistema ([BALBINO, 2020](#); [BRINDEIRO, 2017](#); [BAPTISTA et al., 2022](#)); a infraestrutura física do laboratório (estações de trabalho, dispositivos de RV, triciclo, estimuladores e sensores); e o registro das tentativas de execução. Os instrumentos de análise compreendem a inspeção de repositórios sob os critérios de boas práticas (presença de **README**, licença, histórico de *commits*, organização e nomenclatura), a leitura de código-fonte e a diagramação da arquitetura.

3.5 Delimitação do Escopo

Cabe delimitar o alcance desta etapa. Consoante o caráter preparatório de um primeiro trabalho de conclusão de curso, o escopo aqui é o de *diagnóstico, documentação e proposta* — e não o da implementação e validação experimental das soluções, reservadas à etapa subsequente. Reconhece-se, ademais, que o prazo de execução não permite a reprodução integral de sistemas cuja documentação é ausente ou incompleta; essa própria impossibilidade, contudo, é um dos resultados diagnósticos do trabalho, e não uma limitação a ser omitida.

4 Estudo de Caso A — Realidade Virtual

Este capítulo apresenta o primeiro estudo de caso: o sistema de realidade virtual imersivo do Projeto EMA, materializado no repositório `ema_vr_rehab`. Trata-se de um sistema cujo último desenvolvimento registrado no repositório data de 2019 e que se encontra inativo desde então; o seu desafio de Engenharia de Software é, portanto, o de reativação — recuperar, a partir do que está registrado, um sistema que já funcionou. O capítulo documenta o que o sistema é (Seção 4.1), a sua arquitetura atual (Seção 4.2), a tentativa de reprodução (Seção 4.3) e as limitações específicas identificadas (Seção 4.4).

4.1 Visão Geral do Sistema

O sistema de RV foi concebido para capturar o movimento do paciente por meio de sensores inerciais e reproduzi-lo, em tempo real, em um avatar tridimensional imersivo, fornecendo realimentação visual capaz de aumentar a imersão, o engajamento e a motivação durante as sessões de terapia (BALBINO, 2020). Do ponto de vista da visualização do movimento (Seção 2.6.3), o sistema adota a estratégia baseada em *avatar*: o paciente, imerso pelo capacete de realidade virtual, observa uma representação virtual do próprio corpo que se move em correspondência com os seus movimentos reais. A Figura 4 mostra o sistema em uso por um voluntário com lesão medular.

Figura 4 – Voluntário utilizando o sistema de realidade virtual do Projeto EMA.



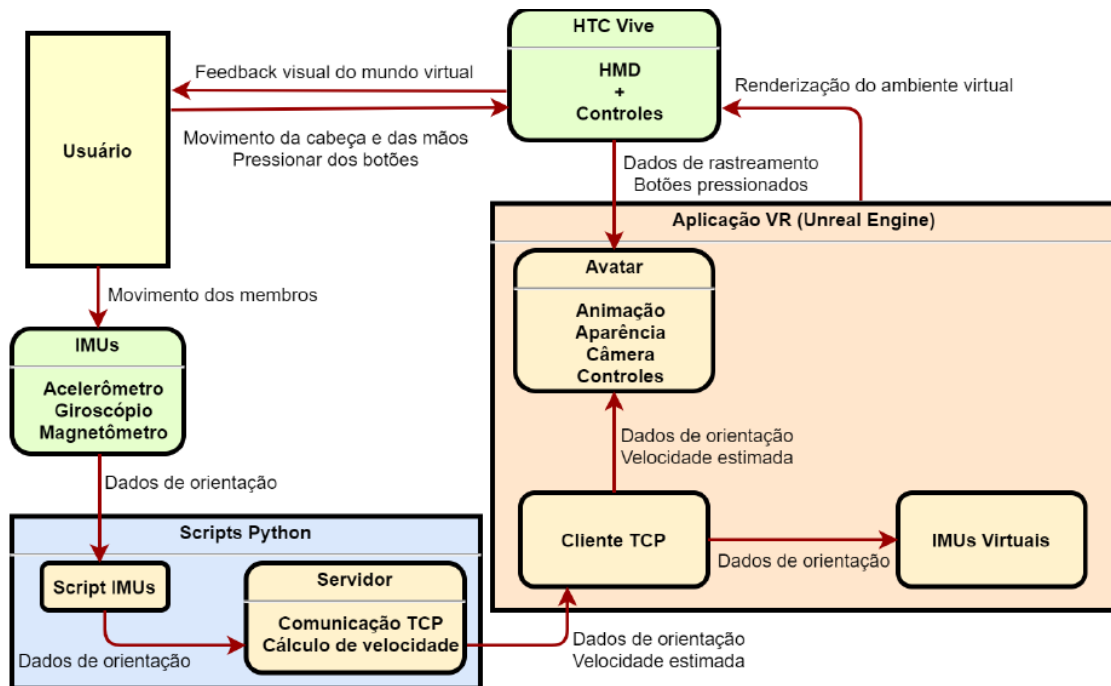
Fonte: [Balbino \(2020\)](#). O voluntário utiliza o HMD e os controles do HTC Vive e uma IMU no braço direito; a visão do ambiente virtual é espelhada no monitor ao lado.

O hardware de imersão empregado é o *HTC Vive* de primeira geração, com rastreamento externo por estações de base e dois controles de movimento (*Wands*). Um ponto que merece registro, por corrigir uma imprecisão recorrente, diz respeito ao escopo do movimento capturado: a análise do repositório e a documentação da fonte mostram que as orientações provenientes dos sensores inerciais são aplicadas tanto aos **membros inferiores** quanto aos **membros superiores** do avatar. Adicionalmente, os braços podem ser controlados pelos controles de movimento do HTC Vive por meio de cinemática inversa, e uma IMU no braço é utilizada para estimar a velocidade de deslocamento nas atividades de marcha.

4.2 Arquitetura Atual (As-Is)

A arquitetura documentada do sistema, reproduzida na Figura 5, organiza-se em três blocos: o usuário e os seus sensores; um conjunto de *scripts* em Python; e a aplicação de realidade virtual construída sobre a *game engine Unreal Engine*. As IMUs capturam o movimento dos membros e um *script* em Python — que inclui um servidor responsável pelo cálculo de velocidade — transmite as orientações à *engine*. Na *engine*, o avatar tem a sua animação, aparência e câmera controladas por esses dados, e o ambiente renderizado é exibido no HMD, fechando o laço de realimentação visual com o usuário.

Figura 5 – Arquitetura do sistema de realidade virtual, conforme documentada na fonte.



Fonte: Balbino (2020).

A análise do código-fonte confirma o desenho documentado, mas revela camadas adicionais que a documentação não explicita — exatamente o tipo de distância entre o *registrado* e o *montado* que este trabalho investiga. Três observações destacam-se. Primeira, quanto à *comunicação*: além do canal por *socket* TCP entre o servidor Python e a *engine* (o mecanismo documentado na Figura 5), o repositório contém ainda uma integração via *plugin ROSIntegration* (uma ponte estilo *rosbridge*), remanescente de uma abordagem anterior. Coexistem, assim, dois mecanismos de comunicação no mesmo código, sem que uma camada única e documentada prevaleça. Segunda, quanto à *localização da lógica*: a aplicação das orientações aos ossos do avatar — o núcleo funcional do sistema — está implementada majoritariamente em *Blueprints*, a linguagem visual da Unreal Engine. Por serem artefatos binários, os *Blueprints* não permitem comparação de versões (*diff*) nem revisão de código no mesmo sentido que o código textual, o que reduz a rastreabilidade e dificulta a manutenção. Terceira, quanto ao *acoplamento*: o tratamento das orientações e a sua conversão em movimento do avatar encontram-se embarcados dentro da própria *engine*, de modo que decisões que deveriam ser independentes — trocar o capacete, migrar de *engine* ou substituir o sensor — repercutem sobre componentes que não deveriam ser afetados, elevando o custo de qualquer evolução.

4.3 Tentativa de Reprodução

A primeira barreira à reprodução é a *ausência de um README* ou de qualquer documentação de instalação no repositório — ausência confirmada por inspeção direta, diferentemente de outros repositórios do laboratório que dispõem de documentação (ver Capítulo 6). Não há indicação da versão da *engine*, das dependências, dos passos de configuração do ambiente nem do procedimento para executar o sistema. A reconstrução do funcionamento depende, assim, de engenharia reversa a partir do código e do histórico de *commits*, agravada pelo fato de parte substancial da lógica residir em *Blueprints* binários. Some-se a isso a dependência de uma geração de hardware de RV descontinuada (o HTC Vive de primeira geração) e a necessidade de verificar a integridade dos modelos e *assets* do avatar — itens cuja disponibilidade só pode ser plenamente confirmada na estação de trabalho do laboratório, etapa reservada à continuidade do trabalho.

4.4 Limitações Específicas Identificadas

Consolidam-se, a seguir, as limitações específicas deste estudo de caso, sem prejuízo das limitações comuns tratadas no Capítulo 6:

- **Ausência de documentação de instalação e uso:** inexistência de README, de guia de dependências e de passo a passo de execução.
- **Lógica central em artefatos binários:** dependência de *Blueprints* para a lógica de aplicação do movimento, o que compromete a rastreabilidade, a revisão e a comparação de versões.
- **Arquitetura monolítica e acoplada:** lógica de aquisição entrelaçada à lógica de interface dentro da *engine*, dificultando a substituição de componentes.
- **Camada de comunicação não unificada:** coexistência de uma ponte via ROSIntegration e de comunicação por *socket* TCP, sem uma camada única e documentada.
- **Dependência de tecnologia descontinuada:** sistema construído sobre uma geração de hardware de RV (HTC Vive de primeira geração) já descontinuada.
- **Risco quanto aos *assets*:** necessidade de verificar, na etapa de reativação, a disponibilidade e a integridade dos modelos e recursos gráficos do avatar.

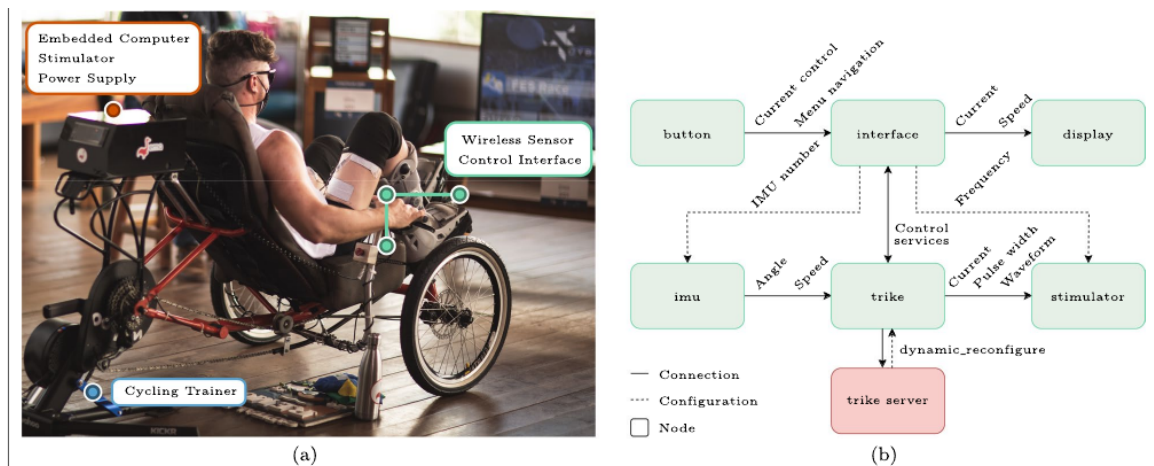
5 Estudo de Caso B — Ciclismo com Eletroestimulação Funcional

Este capítulo apresenta o segundo estudo de caso: o sistema de ciclismo assistido por eletroestimulação funcional (FES *Cycling*) do Projeto EMA. Diferentemente do sistema de realidade virtual, trata-se de um sistema *ativo*, com equipe atualmente trabalhando sobre ele — o que não o isenta, contudo, das mesmas fragilidades de Engenharia de Software. O capítulo documenta o que o sistema é (Seção 5.1), a sua arquitetura atual (Seção 5.2), a tentativa de reprodução (Seção 5.3) e as limitações específicas identificadas (Seção 5.4).

5.1 Visão Geral do Sistema

O sistema controla um triciclo reclinado (*trike*) adaptado, em configuração *tadpole*, no qual a pedalada é produzida pela estimulação elétrica de músculos paralisados ou enfraquecidos do ciclista. Um sensor inercial mede o ângulo do pedal (*crank*) e, a partir dele, uma lógica de controle determina quais grupos musculares estimular e em que instante, de modo a produzir uma pedalada coordenada e segura (BRINDEIRO, 2017; BAPTISTA et al., 2022). O sistema é herdeiro de uma linha de pesquisa consolidada do laboratório, que inclui a participação da equipe na competição internacional CYBATHLON e o emprego de técnicas avançadas de estimulação centradas no usuário (BAPTISTA et al., 2022). A Figura 6 apresenta o sistema e a sua arquitetura de controle, tal como documentados.

Figura 6 — Sistema de ciclismo com FES: (a) montagem física; (b) arquitetura de controle em nós.



Fonte: Baptista et al. (2022).

Conforme documentado, o núcleo de processamento é um computador embarcado (Raspberry Pi), ao qual se conectam um eletroestimulador multicanal, um sensor inercial sem fio que mede o ângulo do *crank* e uma interface humano-máquina (HMI) composta por botões físicos e um display LCD. A interação com o operador e o atleta é mediada por essa HMI.

5.1.1 Dois Repositórios, Duas Gerações

Um aspecto essencial para compreender este estudo de caso — e um achado de diagnóstico em si — é que o sistema existe hoje sob duas gerações tecnológicas, em dois repositórios distintos:

- `ema_fes_cycling` (público) — a implementação **original em ROS 1**, à qual corresponde a arquitetura documentada da Figura 6. Este repositório é razoavelmente documentado: possui `README`, `Wiki` com guias de instalação (para PC e para Raspberry Pi) e de primeiros passos, licença definida e versões marcadas (*releases*). Reflete, porém, uma geração tecnológica anterior.
- `ema_fes_ros2` (privado) — a **reescrita em ROS 2**, que constitui a linha de desenvolvimento ativa. Ao contrário do original, este repositório praticamente não dispõe de documentação: o `README` resume-se a poucas linhas de anotações técnicas, sem guia de instalação ou de execução.

Esse desdobramento evidencia um fenômeno recorrente: a migração de ROS 1 para ROS 2 produziu um novo repositório que *perdeu* a documentação que o original possuía, além de fragmentar a mesma linha de trabalho em dois artefatos com estados de documentação opostos. A tentativa de reprodução descrita adiante refere-se à geração ativa (ROS 2).

5.2 Arquitetura Atual (As-Is)

A arquitetura de controle apoia-se no modelo de nós e tópicos do ROS descrito na Seção 2.5, ilustrado na Figura 6(b). Na versão documentada, os nós dividem-se por função: um nó de botões e um nó de *display* compõem a HMI; um nó de interface interpreta os comandos e gerencia a navegação; um nó de IMU publica o ângulo e a velocidade do *crank*; e um nó central de controle calcula os instantes de estimulação a partir do ângulo e da velocidade e envia ao nó do estimulador os parâmetros de corrente, largura de pulso e forma de onda. Na reescrita ativa em ROS 2, essa lógica de controle está a cargo dos nós `control_node` e `control_shift_node`, ao lado de `imu_node`, `stim_node` e `hmi_node`, preservando a mesma divisão de responsabilidades.

A lógica de decisão do controle apoia-se em faixas angulares: para cada grupo muscular, definem-se um ângulo inicial de estimulação e uma amplitude, de modo que o ângulo final de parada seja `stop_angles = start_angles + angle_range`; verifica-se, a cada instante, se a posição do pedal está na zona de estimulação ou na zona de parada. Por assentar-se no ROS 2, a versão ativa dispõe, em princípio, dos mecanismos de Qualidade de Serviço e de segurança discutidos na Seção 2.5 — como a diferenciação entre telemetria tolerante a perdas e comandos de entrega garantida, e salvaguardas que coloquem o estimulador em modo seguro caso o nó de controle deixe de emitir comandos válidos. Cabe ressaltar, todavia, que o uso pleno dessas salvaguardas figura, no material analisado, mais como *plano* do que como implementação confirmada; a sua efetivação é uma das recomendações do Capítulo 7.

5.2.1 Lacuna entre o Registrado e o Montado

Um achado central deste estudo de caso é a divergência entre a arquitetura documentada e a efetivamente montada. A interface humano-máquina, por exemplo, depende de um microcontrolador (ESP32 ou Arduino) que se comunica com o computador embarcado; o *display* constitui, na prática, um componente que troca dados com esse microcontrolador. Contudo, tais elementos não figuram de forma completa nos diagramas de referência preexistentes, o que caracteriza um diagrama *incompleto* em relação ao hardware real. Essa é uma manifestação concreta da lacuna entre o que está registrado e o que está montado, e um dos pontos que a diagramação padronizada proposta neste trabalho busca corrigir.

5.3 Tentativa de Reprodução

A tentativa de executar a geração ativa (ROS 2, repositório `ema_fes_ros2`) tal como se encontra revelou uma sucessão de obstáculos, aqui registrados como dados de diagnóstico:

- **Ausência de documentação no repositório ativo:** o repositório da reescrita em ROS 2 não traz documentação que oriente a execução, restando a descoberta manual dos modos de funcionamento por tentativa e erro.
- **Falha na execução containerizada:** a tentativa natural de subir o sistema via `docker compose` esbarra na referência a uma imagem específica que não está publicada em lugar algum, obrigando a improvisar com uma versão aproximada.
- **Restrições rígidas de ambiente:** a versão do ROS 2 empregada exige um sistema operacional específico (Ubuntu 22.04, com Python 3.10 e bibliotecas compatíveis), o que torna o ambiente pouco portátil.

- **Hierarquia de inicialização rígida:** existe um arquivo “inicializador” concebido para “facilitar” a partida do sistema, mas que, na prática, dificulta a depuração: os nós ficam à espera de parâmetros de configuração oriundos dessa inicialização e não podem ser executados isoladamente, forçando uma operação de “tudo ou nada”.
- **Código-fonte ausente:** falta um dos códigos mais importantes — o da interface humano-máquina (HMI), que depende apenas de um ESP32 ou Arduino —, cuja ausência impede inclusive a depuração do que ocorre durante o treino.
- **Documentação de infraestrutura incompleta:** a documentação existente menciona um servidor do EMA e recursos de acesso remoto e telemetria que são praticamente impossíveis de reimplementar, por não haver descrição de como configurá-los.
- **Tentativa em plataforma alternativa:** uma tentativa de instalação manual em uma Raspberry Pi 4B (sem Docker, para não onerar o hardware) esbarrou novamente na hierarquia de inicialização, que impediu a execução individual dos nós para fins de depuração.

O conjunto desses obstáculos sustenta a resposta à pergunta-guia do trabalho: na geração ativa e nas condições atuais, o sistema *não* pode ser reproduzido por um novo integrante sem esforço considerável de engenharia reversa e sem acesso a conhecimento tácito não documentado — e isso apesar de a geração anterior (ROS 1) dispor de guias de instalação.

5.4 Limitações Específicas Identificadas

- **Documentação perdida na migração:** a reescrita ativa em ROS 2 não herdou a documentação (README, *Wiki*, guias) que o repositório original em ROS 1 possui.
- **Código-fonte incompleto:** ausência do código da HMI (ESP32/Arduino), essencial para a operação e a depuração do treino.
- **Reprodutibilidade comprometida:** dependência de imagem de contêiner inexistente e de ambiente rígido, sem passo a passo de instalação no repositório ativo.
- **Acoplamento na inicialização:** hierarquia de *launch* que impede a execução isolada de nós, dificultando testes e depuração.
- **Documentação de hardware ausente:** inexistência de um diagrama de hardware que permita replicar o sistema para testes ou manutenções.
- **Infraestrutura não documentada:** servidor e recursos de telemetria/acesso remoto sem documentação de configuração.

- **Incoerência entre diagramas e hardware:** componentes reais (como o *display* conectado ao microcontrolador) ausentes dos diagramas de referência.

6 Diagnóstico Consolidado

Os dois estudos de caso, embora distintos em domínio e em estado de atividade, convergem para um mesmo conjunto de limitações de Engenharia de Software. Este capítulo consolida os problemas *comuns* a ambos, tratando-os de forma unificada. Começa-se pela organização do versionamento (Seção 6.1), passa-se à documentação e à reprodutibilidade (Seção 6.2) e à arquitetura e ao acoplamento (Seção 6.3), e encerra-se com uma síntese da lacuna entre o registrado e o montado (Seção 6.4).

6.1 Organização do Versionamento e dos Repositórios

A inspeção da organização `lara-unb` no GitHub revela um quadro heterogêneo: há repositórios cuidados e repositórios negligenciados, sem uma política uniforme que os distinga. Os principais achados são:

- **Fragmentação da mesma linha de trabalho:** um mesmo sistema pode existir em múltiplos repositórios que representam gerações tecnológicas distintas. O caso mais ilustrativo é o do ciclismo com FES, cuja implementação original em ROS 1 (`ema_fes_cycling`, público e documentado) coexiste com a reescrita ativa em ROS 2 (`ema_fes_ros2`, privado e praticamente sem documentação), sem que a relação entre ambos esteja explicitada.
- **Dispersão do ecossistema:** os subsistemas do Projeto EMA distribuem-se por mais de uma dezena de repositórios de nomes pouco padronizados, agrupando análise de movimento, estimulação, sensores, botões e mensagens sem uma convenção que comunique a sua relação e o seu estado (ativo ou arquivado).
- **Higiene de repositório desigual:** embora a maioria dos repositórios possua README — em uma amostragem de 89 repositórios da organização, 77 dispunham de algum README —, a sua qualidade é irregular, e justamente alguns dos artefatos mais relevantes carecem de documentação adequada: é o caso do sistema de realidade virtual (`ema_vr_rehab`, sem README), da reescrita ativa do ciclismo (`ema_fes_ros2`, com README mínimo) e de diversos módulos de baixo nível (drivers de IMU, de estimulador e de sensor de força, entre outros, sem README).
- **Padronização inconsistente de metadados:** a definição de licença, de descrição e o uso de recursos como *releases* variam de repositório para repositório — alguns, como o `ema_fes_cycling`, adotam licença explícita e versões marcadas, enquanto outros, inclusive os ativos, não trazem licença nem descrição.

O efeito combinado desses fatores é a perda de rastreabilidade: torna-se difícil, mesmo para um integrante experiente, determinar qual repositório contém a versão vigente de um sistema, qual o seu estado e como executá-lo. Trata-se, paradoxalmente, de um dos problemas mais custosos e, ao mesmo tempo, dos mais simples de mitigar com convenções básicas.

6.2 Documentação e Reprodutibilidade

Mais do que a mera presença de um `README`, o que os dois casos revelam é uma *insuficiência de documentação orientada à reprodução*, concentrada precisamente nos artefatos ativos. Nenhum dos dois sistemas, na sua geração vigente, oferece o conjunto mínimo que permitiria a um novo integrante instalá-lo e executá-lo de forma autônoma: faltam guias de instalação atualizados, especificação de dependências e de ambiente e passo a passo de execução. É sintomático que, no caso do ciclismo, a documentação existente esteja associada à geração *anterior* (ROS 1), ao passo que a geração *ativa* (ROS 2) — aquela que um novo integrante de fato utilizaria — seja a desprovida de guias. Como registrado nos Capítulos 4 e 5, a tentativa de reprodução esbarra sistematicamente em conhecimento tácito não registrado — versões de *engine*, lógica embarcada em *Blueprints*, imagens de contêiner inexistentes, parâmetros de inicialização e detalhes de infraestrutura. O resultado é uma *reprodutibilidade comprometida*: o objetivo de que um novo integrante consiga, a partir do acesso ao repositório, colocar o sistema em funcionamento não é hoje atingível sem esforço considerável de engenharia reversa.

6.3 Arquitetura e Acoplamento

Ambos os sistemas exibem forte acoplamento entre a lógica de aquisição de dados e a lógica de interface, ainda que com naturezas distintas. No caso da RV, o acoplamento manifesta-se na incorporação da lógica de aquisição à própria *game engine*, com parte substancial embarcada em *Blueprints* binários; no caso do ciclismo, na hierarquia de inicialização que impede a execução isolada de nós. Em ambos, o acoplamento eleva o custo de evolução e dificulta a substituição de componentes e a depuração. Reconhece-se, contudo, uma assimetria favorável ao sistema de ciclismo: por apoiar-se no ROS 2 e no seu modelo de nós e tópicos (Seção 2.5), ele já dispõe de uma base arquitetural modular sobre a qual o desacoplamento pode ser construído — desde que a rigidez da inicialização seja tratada.

6.4 Síntese: a Lacuna entre o Registrado e o Montado

O fio condutor do diagnóstico é a distância entre aquilo que está *registrado* — em artigos, diagramas e trabalhos anteriores — e aquilo que está de fato *montado e em operação*. Essa lacuna assume três formas recorrentes: (i) *documentação ausente ou desatualizada*, quando não há registro do que existe hoje, ou quando o registro se refere a uma geração anterior; (ii) *documentação incompleta ou incoerente*, quando o registro existe mas não corresponde ao sistema real — como no diagrama que omite o *display* conectado ao microcontrolador; e (iii) *código ausente*, quando o próprio artefato de software que deveria existir não é encontrado — como o código da HMI do sistema de ciclismo. A Tabela 1 sintetiza os achados por estudo de caso.

Tabela 1 – Síntese das limitações identificadas por estudo de caso.

Limitação	RV	Ciclismo (FES)
Documentação de instalação insuficiente (repositório ativo)	Sim	Sim
Fragmentação/inconsistência no versionamento	Sim	Sim
Acoplamento entre aquisição e interface	Sim	Sim
Reprodutibilidade comprometida	Sim	Sim
Lógica central em artefatos binários (<i>Blueprints</i>)	Sim	—
Dependência de tecnologia descontinuada (HTC Vive)	Sim	—
Código-fonte ausente (HMI)	—	Sim
Diagrama incoerente com o sistema real	Sim	Sim
Infraestrutura não documentada (servidor)	—	Sim

Fonte: elaborado pelos autores.

Esse mapeamento fundamenta as recomendações apresentadas no capítulo seguinte, cuja lógica é atacar primeiro as limitações mais básicas e de maior alcance — versionamento e documentação — antes das intervenções estruturais mais profundas.

7 Recomendações e Caminho de Reestruturação

A partir do diagnóstico consolidado no Capítulo 6, este capítulo reúne as recomendações de Engenharia de Software e esboça um caminho de reestruturação para os sistemas do Projeto EMA. A lógica que organiza as recomendações é a de atacar primeiro as limitações mais básicas e de maior alcance — aquelas que, com esforço reduzido, mais aproximam o laboratório do objetivo de que um novo integrante consiga reproduzir e evoluir os sistemas — para só então avançar sobre as intervenções estruturais mais profundas. Embora vários exemplos remetam ao sistema de ciclismo com FES, por ser a frente ativa, os princípios aplicam-se igualmente ao sistema de realidade virtual.

7.1 Padronização do Versionamento e dos Repositórios

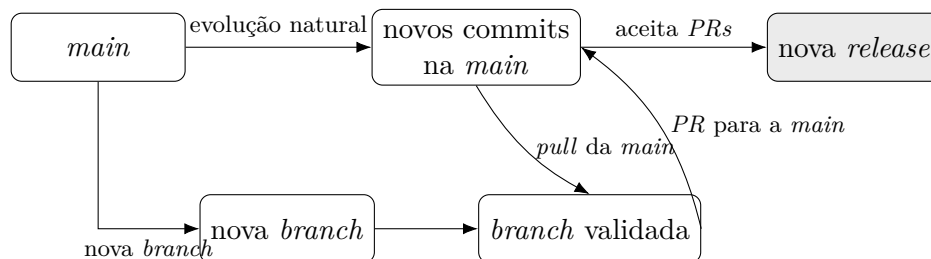
O primeiro eixo de recomendações trata da governança do código-fonte, cujo estado atual foi diagnosticado na Seção 6.1. Propõem-se as seguintes medidas:

- **Convenção de nomenclatura e consolidação:** padronizar os repositórios sob uma estrutura clara de pacotes ROS 2 e identificar, para cada sistema, o *repositório canônico* — no caso do ciclismo, o `ema_fes_ros2` —, eliminando duplicações e fragmentações de código.
- **Ciclo de vida do repositório:** arquivar explicitamente código obsoleto e testes isolados de bancada, migrando para a árvore principal de desenvolvimento apenas os módulos funcionais já validados.
- **Gerenciamento de escopo:** utilizar ativamente os recursos do GitHub — *Issues* para o rastreamento de defeitos (por exemplo, falhas de comunicação serial), *Releases* para o versionamento de *builds* estáveis do firmware e a divisão em equipes com proteção das ramificações principais. Recomenda-se, ainda, uma hierarquia de aprovação — com a participação dos professores — para que alterações sejam promovidas à ramificação principal.
- **Template mínimo de repositório:** exigir uma estrutura de entrada padronizada, contendo um arquivo `README.md` técnico, a atribuição de uma licença de software e um arquivo `.gitignore` configurado para ignorar os artefatos gerados pela compilação do ROS 2 (as pastas `build/`, `install/` e `log/`).

- **Redundância e preservação:** adotar cópias de segurança em plataformas complementares ao repositório principal, gerando redundância que reduza a dependência de um único provedor e proteja o acervo do projeto contra eventuais perdas.

Um ponto merece detalhamento por incidir diretamente sobre a qualidade do código ao longo do tempo: a *organização dos commits na ramificação principal segundo práticas de desenvolvimento de software livre*. Em vez de escrever diretamente na ramificação principal, propõe-se que todo novo trabalho ocorra em uma ramificação própria e só retorne à principal após verificação, validação e resolução de conflitos, por meio de uma requisição de incorporação (*pull request*). Esse fluxo, ilustrado na Figura 7, reduz a quantidade de defeitos reinseridos e o retrabalho de refatoração a cada iteração.

Figura 7 – Fluxo de versionamento proposto para a ramificação principal.



Fonte: elaborado pelos autores.

7.2 Documentação Mínima e Onboarding

O segundo eixo define um conjunto mínimo de documentação por sistema, tratado como código (*docs-as-code*) e versionado junto ao repositório, orientado pelo arcabouço Diátaxis (Seção 2.3):

- **Estrutura baseada em Diátaxis:** elaborar arquivos em Markdown com uma visão geral do ecossistema, os requisitos de infraestrutura e um passo a passo *reproduzível* para a inicialização do sistema como um todo.
- **Guia de *onboarding* e *setup*:** fornecer instruções claras para a preparação do ambiente (por exemplo, Ubuntu Server 22.04 LTS em uma Raspberry Pi), a inclusão do usuário no grupo `dialout` para acesso às portas seriais e a instalação das dependências do ROS 2.
- **Registro de Decisões de Arquitetura (ADR):** documentar formalmente as escolhas críticas de projeto — como a adoção do DDS para a descentralização dos nós e a opção pela persistência local de configurações em vez de um banco de dados tradicional —, preservando o contexto e as consequências de cada decisão.

7.3 Diagramação Padronizada da Arquitetura

O terceiro eixo trata da produção e da manutenção de um conjunto de diagramas coerentes, corrigindo as lacunas de modelagem identificadas:

- **Grafo de nós e tópicos do ROS 2:** representar com clareza o fluxo de publicação/assinatura entre os nós de controle e de aquisição de dados.
- **Mapeamento de hardware e infraestrutura:** diagramar explicitamente o ciclo de hardware, incluindo as regras de *kernel* do Linux (UDEV) que criam vínculos simbólicos persistentes para as portas seriais (por exemplo, `/dev/ttyUSB0` e `/dev/ttyACM0`), evitando ambiguidades na identificação dos dispositivos.
- **Representação completa da HMI:** criar e diagramar os fluxos de hardware da interface humano-máquina — corrigindo a omissão identificada no Capítulo 5 —, de modo a explicitar os pontos de conexão e a facilitar a reimplementação e a evolução do conjunto hardware/software.

7.4 Modularização e Desacoplamento

O quarto eixo reúne diretrizes para isolar o processamento de baixo nível das camadas de interação com o operador, atacando o acoplamento diagnosticado na Seção 6.3:

- **Flexibilização da inicialização:** estruturar o pacote lógico de modo a permitir que nós críticos sejam executados e testados de forma unitária (por exemplo, `ros2 run ema_fes_cycling imu_node`), superando a atual operação de “tudo ou nada”.
- **Isolamento de código e testes de bancada:** implementar, no script gerenciador do operador (`menu.sh` / `menu.py`), rotinas que permitam a execução independente de *scripts* puramente em Python (`python3 -I`), temporariamente desacoplados do *middleware*, para tarefas como a calibração de sensores.
- **Camada de comunicação padronizada:** adotar um contrato de dados estrito no pacote de interfaces (`ema_custom_interfaces`, com mensagens `.msg` e serviços `.srv`), blindando os nós de controle contra alterações nos firmwares dos microcontroladores (ESP-IDF / Arduino).

7.5 Caminho para a Integração (Trabalho Futuro)

O quinto eixo esboça o caminho que prepara as etapas de expansão subsequentes, objeto do trabalho de conclusão de curso seguinte:

- **Convergência de subsistemas:** planejar a fusão modular entre a plataforma de ciclismo com FES e os módulos de realidade virtual, unindo as duas frentes deste trabalho em um único ecossistema de reabilitação.
- **Arquitetura *plug-and-play*:** reestruturar o barramento de tópicos do ROS 2 para suportar a adição de novas modalidades de estímulo ou de novos sensores cinemáticos de forma transparente.
- **Objetivo subsequente:** consolidar essa infraestrutura de software — padronizada, distribuída e resiliente a falhas — como a base estável e homologada para o desenvolvimento posterior.

8 Considerações Finais

Este trabalho partiu de uma constatação simples e, ao mesmo tempo, incômoda: os sistemas do Projeto EMA, embora tenham cumprido os seus objetivos de pesquisa, não podem hoje ser reproduzidos por um novo integrante sem esforço considerável. A pergunta que orientou a investigação — de que forma o instrumental da Engenharia de Software pode ser aplicado ao diagnóstico, à documentação e à reestruturação desses sistemas — foi respondida por meio de dois estudos de caso complementares, o sistema de realidade virtual e o sistema de ciclismo com eletroestimulação funcional, examinados sob uma postura de consultoria.

8.1 Contribuições

As principais contribuições deste trabalho são de natureza diagnóstica e metodológica. Documentou-se e diagramou-se a arquitetura atual de dois sistemas até então pouco descritos, reconstruindo-a por análise de código e engenharia reversa. Mapeou-se, de forma sistemática, a lacuna entre o que está *registrado* e o que está de fato *montado* — desde a documentação perdida na migração de ROS 1 para ROS 2, no caso do ciclismo, até a lógica embarcada em *Blueprints* e a coexistência de mecanismos de comunicação, no caso da realidade virtual. A partir desse diagnóstico, consolidaram-se recomendações de Engenharia de Software organizadas por prioridade — versionamento, documentação, diagramação, modularização e caminho de integração —, que constituem um roteiro acionável para as etapas seguintes. O conjunto entrega ao laboratório um retrato fiel do seu ponto de partida e um plano para evoluir a partir dele.

8.2 Limitações do Trabalho

Reconhecem-se as limitações desta etapa. Consoante o seu caráter preparatório, o escopo foi o de diagnóstico, documentação e proposta — não o da implementação e da validação experimental das soluções, reservadas à etapa subsequente. Ademais, o prazo de execução não permitiu a reprodução integral de sistemas cuja documentação é ausente ou incompleta; essa própria impossibilidade, contudo, não é uma omissão, mas um dos resultados diagnósticos do trabalho, e reforça a pertinência das recomendações apresentadas.

8.3 Trabalhos Futuros

Como continuidade natural, aponta-se a implementação das recomendações do Capítulo 7: a padronização dos repositórios e da documentação, a reativação efetiva do sistema de realidade virtual, a reestruturação e a modularização do sistema de ciclismo com FES e, ao final, a convergência das duas frentes em um ecossistema de reabilitação integrado e *plug-and-play*. Consolidada essa base de software — padronizada, documentada e resiliente —, o laboratório estará apto a retomar, com segurança e continuidade, a evolução de tecnologias cujo propósito último é ampliar a autonomia e a qualidade de vida de pessoas em reabilitação.

Referências

- BALBINO, H. d. S. e. S. *Desenvolvimento de sistema de realidade virtual imersivo para reabilitação de marcha usando estimulação elétrica*. 743 p. Dissertação (Dissertação (Mestrado em Engenharia de Sistemas Eletrônicos e de Automação)) — Universidade de Brasília, Brasília, 2020. Publicação PGEA.TD-743/2020. Orientador: Prof. Dr. Antonio Padilha Lanari Bó. Citado 5 vezes nas páginas 16, 30, 32, 33 e 34.
- BAPTISTA, R. S. et al. User-centered design and spatially-distributed sequential electrical stimulation in cycling for individuals with paraplegia. *Journal of NeuroEngineering and Rehabilitation*, BioMed Central, v. 19, n. 1, p. 45, 2022. Citado 4 vezes nas páginas 16, 27, 30 e 36.
- BECK, K. *Test-driven development: by example*. Boston: Addison-Wesley, 2003. ISBN 978-0-321-14653-3. Citado na página 22.
- BRINDEIRO, G. A. *Software embarcado de controle para triciclo assistido por estimulação elétrica*. Dissertação (Dissertação (Mestrado em Engenharia de Sistemas Eletrônicos e de Automação)) — Universidade de Brasília, Brasília, 2017. Publicação PPGEA 661/2017 DM. Orientador: Prof. Dr. Antônio Padilha Lanari Bó. Citado 4 vezes nas páginas 16, 27, 30 e 36.
- BROWN, S. *The C4 model for visualising software architecture*. 2024. <<https://c4model.com>>. Acesso em: jul. 2026. Citado na página 23.
- DONATI, A. R. C. et al. Long-term training with a brain-machine interface-based gait protocol induces partial neurological recovery in paraplegic patients. *Scientific Reports*, Nature Publishing Group, v. 6, p. 30383, 2016. Citado na página 27.
- Ferreira dos Santos, L. et al. Movement visualisation in virtual reality rehabilitation of the lower limb: a systematic review. *BioMedical Engineering OnLine*, BioMed Central, v. 15, n. Suppl 3, p. 144, 2016. Citado 2 vezes nas páginas 26 e 27.
- FOWLER, M. *Refactoring: improving the design of existing code*. 2. ed. Boston: Addison-Wesley, 2018. ISBN 978-0-13-475759-9. Citado na página 21.
- KLEIM, J. A.; JONES, T. A. Principles of experience-dependent neural plasticity: implications for rehabilitation after brain damage. *Journal of Speech, Language, and Hearing Research*, American Speech-Language-Hearing Association, v. 51, n. 1, p. S225–S239, 2008. Citado na página 25.
- LAVER, K. E. et al. Virtual reality for stroke rehabilitation. *Cochrane Database of Systematic Reviews*, John Wiley & Sons, n. 11, p. CD008349, 2017. Citado na página 26.
- LEE, J. et al. Effects of immersive virtual reality training on balance, gait and mobility in older adults: a systematic review and meta-analysis. *Gait & Posture*, Elsevier, v. 110, p. 129–137, 2024. Citado na página 27.

- LOUREIRO, M. et al. Analysis of gait kinematics in smart walker-assisted locomotion in immersive virtual reality scenario. *Sensors*, MDPI, v. 24, n. 17, p. 5534, 2024. Citado 2 vezes nas páginas 14 e 18.
- MACCHITELLA, L. et al. A narrative review of the use of a cutting-edge virtual reality rehabilitation technology in neurological and neuropsychological rehabilitation. *NeuroRehabilitation*, IOS Press, v. 53, n. 4, p. 439–457, 2023. Citado 2 vezes nas páginas 18 e 25.
- MILGRAM, P.; KISHINO, F. A taxonomy of mixed reality visual displays. *IEICE Transactions on Information and Systems*, Institute of Electronics, Information and Communication Engineers, E77-D, n. 12, p. 1321–1329, 1994. Citado na página 26.
- MIRELMAN, A. et al. Addition of a non-immersive virtual reality component to treadmill training to reduce fall risk in older adults (V-TIME): a randomised controlled trial. *The Lancet*, Elsevier, v. 388, n. 10050, p. 1170–1182, 2016. Citado na página 27.
- NYGARD, M. *Documenting architecture decisions*. 2011. <<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>>. Acesso em: jul. 2026. Citado na página 23.
- Open Robotics. *ROS 2 documentation*. 2024. <<https://docs.ros.org/en/humble>>. Acesso em: jul. 2026. Citado na página 24.
- PROCIDA, D. *Diátaxis: a systematic framework for technical documentation authoring*. 2023. <<https://diataxis.fr>>. Acesso em: jul. 2026. Citado na página 23.
- RAJASHEKAR, D. et al. Technological advances in stroke rehabilitation: robotics and virtual reality. *Physical Medicine and Rehabilitation Clinics of North America*, Elsevier, v. 35, n. 2, p. 383–398, 2024. Citado 2 vezes nas páginas 14 e 25.
- SKARBEZ, R.; SMITH, M.; WHITTON, M. C. Revisiting Milgram and Kishino's reality-virtuality continuum. *Frontiers in Virtual Reality*, Frontiers Media, v. 2, p. 647997, 2021. Citado na página 26.
- SLATER, M.; WILBUR, S. A framework for immersive virtual environments (FIVE): speculations on the role of presence in virtual environments. *Presence: Teleoperators and Virtual Environments*, MIT Press, v. 6, n. 6, p. 603–616, 1997. Citado na página 25.
- SOMMERVILLE, I. *Software Engineering*. 9. ed. Boston: Pearson, 2011. ISBN 978-0-13-703515-1. Citado 2 vezes nas páginas 17 e 20.
- TAKEUCHI, N.; IZUMI, S.-I. Rehabilitation with poststroke motor recovery: a review with a focus on neural plasticity. *Stroke Research and Treatment*, Hindawi, v. 2013, p. 128641, 2013. Citado na página 25.

Apêndices

APÊNDICE A – Inventário dos Repositórios do Laboratório no GitHub

Este apêndice detalha o levantamento da organização `lara-unb` no GitHub que fundamenta o diagnóstico apresentado na Seção 6.1. Em uma amostragem de 89 repositórios da organização, 77 (aproximadamente 87%) dispunham de algum arquivo `README`; a qualidade, contudo, é desigual, e alguns dos artefatos mais relevantes — inclusive os ativos — carecem de documentação adequada. A Tabela 2 resume os repositórios diretamente relacionados aos dois estudos de caso e a módulos representativos do ecossistema, com o seu estado de documentação e de versionamento.

Tabela 2 – Repositórios do ecossistema EMA e o seu estado de documentação e versionamento.

Repositório	Visib.	README	Papel / observação
<code>ema_fes_cycling</code>	Público	Sim	Ciclismo com FES (ROS 1): <i>Wiki</i> com guias, licença Apache 2.0 e <i>releases</i> .
<code>ema_fes_ros2</code>	Privado	Mínimo	Reescrita ativa em ROS 2: sem guia, sem licença e sem <i>releases</i> .
<code>ema_vr_rehab</code>	Público	Não	Sistema de realidade virtual (Unreal): sem <code>README</code> nem licença; último commit em 2019.
<code>ema_common_msgs</code>	Público	Sim	Mensagens ROS compartilhadas do projeto.
<code>hasomed_rehastim_stimulator</code>	Público	Sim	Driver do eletroestimulador (dependência).
<code>yostlabs_3space_imu</code>	Público	Sim	Driver do sensor inercial (dependência).
<code>Py_imu</code> , <code>Py_stimulator</code> , <code>Py_forcesensor</code>	Público	Não	Módulos de baixo nível, sem documentação.

Fonte: elaborado pelos autores a partir da inspeção da organização `lara-unb` (jul. 2026).

O quadro evidencia o padrão discutido no Capítulo 6: a documentação existente tende a concentrar-se em repositórios de gerações anteriores, ao passo que os artefatos ativos — como a reescrita em ROS 2 do ciclismo e o sistema de realidade virtual — são justamente os que menos dispõem de guias de instalação, licença e histórico de versões inteligível.